

Session 2: Iteration and Custom Functions

Loops, map(), apply(), and Reusable Functions

Karry Jiao

dj398@cam.ac.uk

[KJ Github](#)

Oct. 2026

Iteration and custom functions

Vectorization, loops, and `map()`

Use the simplest appropriate tool:

Tool	Best suited to	Typical output
Vectorized operation	an operation already defined for a whole vector	vector
<code>for</code> loop	explicit control over steps or side effects	whatever you build or change
<code>purrr::map()</code>	applying one function consistently to each element	list

```
x ← c(1.2, 2.5, 9.1, 5.8)
```

```
round(x)           # vectorized: simplest here  
map(x, round)      # iteration with a list result
```

`map()` is not automatically parallel and is not always faster; its main benefit is a clear, consistent functional pattern.

Map

- R, however, has a set of functions that allows users to iterate over an object **without writing an explicit loop**
- In this training we'll introduce `map()`. It is a function part of `purrr`, a package that contains tools for functional programming
- Also, in case you have not noticed yet: **R is vectorized!** this means that many operations are applied element-wise by default so you don't have to code loops to apply them to each element of a vector or dataframe

Map

- To use `map()`, you need to load the package `purrr`
- The basic syntax of `map()` is:

`map(X, function, ...)`: applies `function` to each of the elements of `x`. If `x` is a dataframe then `function` is applied column-wise while if it's a vector or a list it is applied item-wise. The output of `map()` is always a list with the results.

- **X**: a dataframe, matrix or vector the function will be applied to
- **function**: the name of the function you want to apply to each of the elements of `x`

Map

```
# Round the values of the following vector  
x ← c(1.2, 2.5, 9.1, 5.8)  
map(x, round)    # Returns a list; same as x %>% map(round)  
round(x)         # since R is vectorized, this also works
```

A loop and `map()` compared

Explicit loop

```
means_loop ← numeric(ncol(df))

for (i in seq_along(df)) {
  means_loop[i] ← mean(df[[i]])
}
```

Functional iteration

```
means_map ← map(df, mean)

# Return a numeric vector
means_dbl ← map_dbl(df, mean)
```

Preallocating the loop output is preferable to repeatedly using `append()`.

The `apply` family

Base R also provides functions for applying the same operation repeatedly.

Function	Iterates over	Typical output
<code>lapply()</code>	elements or data-frame columns	list
<code>sapply()</code>	elements or data-frame columns	simplified vector or matrix when possible
<code>apply()</code>	rows or columns of a matrix-like object	vector, matrix, or array

Use `purrr::map()` when you want predictable output rules and tidyverse-style functional programming; use the base functions when they express the task clearly.

sapply(): apply and simplify

```
# Apply round() to every value
```

```
sapply(c(1.2, 2.5), round)
```

```
# Return the first observation of every column
```

```
sapply(whr, head, 1)
```

Basic syntax:

```
sapply(X, FUN, ... )
```

Unlike `map()`, which consistently returns a list, `sapply()` attempts to simplify its result.

`apply()`: rows or columns

```
mat <- matrix(  
  c(1, 24, 9, 6, 9, 4, 2, 74, 2),  
  nrow = 3  
)  
  
apply(mat, MARGIN = 1, FUN = mean) # row means  
apply(mat, MARGIN = 2, FUN = mean) # column means
```

- `MARGIN = 1`: operate over rows.
- `MARGIN = 2`: operate over columns.

Be careful with mixed-type data frames: `apply()` may first coerce them to a matrix and therefore to a common type.

Exercise: calculate the row maximum

Select `freedom` and `happiness_score`, then calculate the maximum across the two variables for every row.

```
row_max <- whr %>%  
  select(freedom, happiness_score) %>%  
  apply(MARGIN = 1, FUN = max, na.rm = TRUE)
```

Here `MARGIN = 1` tells `apply()` to perform the calculation separately for each row.

Writing a custom function

```
zscore <- function(x) {  
  x_mean <- mean(x, na.rm = TRUE)  
  x_sd   <- sd(x, na.rm = TRUE)  
  
  z <- (x - x_mean) / x_sd  
  return(z)  
}
```

Anatomy:

```
function_name <- function(inputs) {  
  operations  
  return(output)  
}
```

Avoid naming objects `mean` or `sd`, because those names already belong to functions.

Exercise: combine selection, mapping, and a function

1. Select `health_life_expectancy` and `freedom` from `whr`.
2. Apply `zscore()` to both columns.
3. Return a data frame containing the standardized variables.

```
z_scores <- whr %>%  
  select(health_life_expectancy, freedom) %>%  
  map_dfc(zscore)  
  
names(z_scores) <- c("hle_z", "freedom_z")
```

For modern `purrr`, `map_dfc()` may issue a supersession notice; an alternative is `map(... , zscore) >> list_cbind()`.

Thank you!